IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 26 April 2025, accepted 28 June 2025, date of publication 3 July 2025, date of current version 21 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3585746

RESEARCH ARTICLE

SCiMA: A Systolic CiM-Based Accelerator With a New Weight Mapping for CNNs–A Virtual Framework Approach

REZGAR SADEGHI, (Member, IEEE), AND ZAINALABEDIN NAVABI, (Life Senior Member, IEEE)
School of Electrical and Computer Engineering, College of Engineering, University of Tehran, Tehran 1439957131, Iran
Corresponding author: Zainalabedin Navabi (navabi@ut.ac.ir)

ABSTRACT Efficient execution of deep convolutional neural networks (DCNNs) on von Neumann platforms has significantly encountered the memory wall. Computation-in-Memory (CiM) aims to integrate computation and storage by performing analog dot product operations within memory cells. Despite advancements, CiM-based architectures still face bottlenecks in transferring input and partial result data to and from CiM units. To address this, a programmable CiM-based Systolic Accelerator (SCiMA) is proposed, featuring an array of Tiles with deep systolic dataflow within architectural levels, providing a scalable and efficient dataflow for DCNN inference. To reduce data movement more efficiently for convolutional layers, which are feature-dominant, proposed inter- and intra-crossbar weight mapping strategies aim to minimize Tile buffering requirements and enhance data reuse. For dense layers, which have the highest weight dominance, Sparse SCiMA leverages structured pruning to exploit sparsity and enhance area efficiency. Additionally, a virtual SystemC-Python framework is introduced to compile CNN applications onto SCiMA and to explore and evaluate architectural, power, and timing aspects using back-annotated circuit-level information. This framework leveraged matrix-based emulation of the analog computations to calculate the impact of non-idealities in results, thus enabling efficient emulation. Experimental results demonstrate that the proposed CiM-based systolic dataflow and weight mapping for convolutional layers improve computation cycles and data movement by 62% and 57%, respectively, compared to the conventional shared bus structure and weight mapping used in state-of-the-art architectures. Overall, SCiMA improves throughput and power efficiency by 38% and 42% on average, respectively, compared to other architectures. Sparse SCiMA reduces the area and power overhead by 43% and 40% respectively compared to SCiMA.

INDEX TERMS Systolic architecture, computation-in-memory (CiM), weight mapping, convolutional layer, dense layer, memristor crossbar, SCiMA, virtual framework, SystemC.

I. INTRODUCTION

The market size of the big data industry is growing steadily and its applications are permeating every day of our lives. DCNNs are big data algorithms that have shown remarkable breakthroughs in various applications such as image classification and speech recognition. Von-Neumann computing faces a memory bandwidth and energy bottleneck due to the separation of processing and memory [1]. Various accelerators with custom dataflow, compression, and reuse techniques have been proposed [2], but the reliance on off-chip memory (DRAM) still limits bandwidth [1]. Near-memory processing integrates logic within memory to reduce data movement, yet computing and storage remain separate, creating a major throughput bottleneck [3]. Another solution is CiM which brings closer computation and memory parts and increases memory bandwidth by doing computations inside memory. CiM provides high-performance parallelism

The associate editor coordinating the review of this manuscript and approving it for publication was Harikrishnan Ramiah.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

for dot product operation of neural networks and makes it possible to mimic the genuine functionality of neurons by considering memory cells as synapses and applying input activations to multiple rows of memory [1]. For the functioning of CiM units for DCNNs, CiM-based architectures consist of a network of these units alongside a structured system of communication channels. This setup is essential to transmit input vector data onto the crossbar and to gather the incremental output results.

Although there have been significant advancements for newly introduced architectures, the necessity to buffer data within architectural layers remains, resulting in data transfer and storage becoming the predominant factors in energy and throughput [4]. Aiming to minimize data movement, this paper explores various dimensions including dataflow architecture, buffer requirements, data reuse strategies, and sparsity. By optimizing these aspects, the study seeks to enhance computational efficiency and reduce the overhead associated with memory access and data movement. Additionally, a virtual framework is introduced to facilitate hardware compilation and system-level exploration, enabling evaluation of hardware performance and power and assessment of impact of analog non-idealities. Paper contributions are as follows:

- This work firstly introduces a programmable systolic CiM-based accelerator (SCiMA) tailored for DCNNs. SCiMA is a pipelined design that consists of a matrix of Tiles linked via a Network-On-Chip (NOC). Deep systolic dataflow within architectural levels circumvents concentrated movement schemes and rhythmically distributes input and output data across CiM units. Tiles can be programmed to manage intra- and inter-Tile data movement for various matrix multiplication tasks.
- To reduce the volume of activation transfer into Tiles, this research utilizes a kernel-major inter-crossbar weight mapping (KM-InterCWM) for convolution layers. This approach aims to confine computations within a Tile to specific channels of input feature map, thereby diminishing the buffer needs and data movement. In addition, this study proposes a column-major intra-crossbar weight mapping (CM-IntraCWM) to facilitate the datareuse of overlapped consecutive windows, aiming to minimize data transfer to the basic Vector Matrix Multiplication (VMM) units.
- For dense layers which are the most weight-dominant layers in DCNNs and account for approximately 80% of the total weight parameters, this work adopts structured pruning techniques to reduce area overhead. Structured pruning eliminates the least effective rows and columns from matrix partitions, creating a smaller matrix that is mapped onto the same hardware while avoiding the area overhead of performing sparse matrices.
- A system-level virtual framework is presented, consisting of four main components: Firstly, an object-oriented, Python-based framework is presented that facilitates structured compilation, programming, performance evaluation, and analog emulation of CiM-based hardware. Secondly, a mapping algorithm is presented to compile DCNN applications onto the SCiMA by generating configuration parameters for Tiles and providing a systolic dataflow. Thirdly, to expedite the assessment of the suggested hardware under actual data workloads and traffic conditions, this paper introduces a structured SystemC model of the hardware. Finally, given the large number of parameters in DCNNs, directly emulating the hardware and analog computations of all ReRAM memories would be computationally expensive. To mitigate this, a matrix-based modeling approach is adopted for ReRAM arrays, which significantly accelerates the simulations for the assessment of hardware reliability in the presence of analog non-idealities.

The structure of the paper is as follows. Section II provides an overview of CNNs and the concept of CiM. Section III surveys the existing literature on CiM-based architectures and the various approaches to weight mapping. Section IV details the proposed architecture and dataflow. Section V illustrates the weight mapping strategy and pruning of dense layers. Section VI discusses the virtual framework for evaluation of the proposed hardware. Section VII presents and examines the simulation results. Finally, section VIII concludes the paper.

II. BACKGROUND

A. CONVOLUTIONAL NEURAL NETWORKS

CNNs consist of convolution, classification, and max pooling layers, as shown in Figure 1(a). The early convolution layers apply filters to the input feature map (IFM) to generate output feature maps (OFM), capturing edges, objects, and features. During training, the network learns optimal filter weights for feature detection. Max pooling layers reduce IFM dimensions by selecting maximum values from subsets, removing less significant details. Finally, the classifier, using fully connected layers, processes the extracted features to perform image recognition in the last layers.

The functionality of 3-D convolution layer has been shown in Figure 1(b). This layer has (K) filters and takes the output from the preceding layer as its IFM with dimension ($H \times W \times C$) and generates an OFM to serve as the input for the subsequent layer. Each filter comprises C kernels, corresponding to the number of input channels, with each kernel having dimensions of ($r \times s$). OFM is computed by performing a dot product between the filter and the corresponding IFM window, which is determined by sliding the filter over the IFM. Each filter's convolution over the IFM with stride 1 yields one layer of the OFM with dimensions ($H \times W$), resulting in an overall OFM dimension of ($H \times W \times K$).

B. COMPUTATION-IN-MEMORY

CiM is an integrated platform that merges memory and computational components to overcome the memory bandwidth

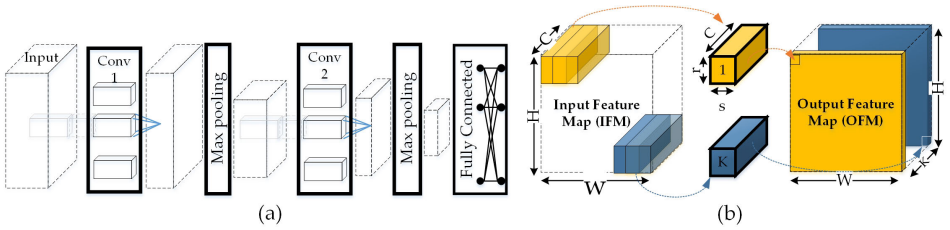


FIGURE 1. (a) Convolutional neural network including convolutional, dense, and max pooling layers, (b) Convolution operation as computationally dominant operation of CNNs.

limitations inherent in Von-Neumann architectures. There are two primary strategies for developing these platforms. The first strategy involves upgrading existing DRAM and SRAM cells to enable certain logical and arithmetic operations directly. These a operations can be performed between the data memory rows [7] or between the memory cells and the inputs applied to the memory crossbar [6]. The second strategy employs Non-Volatile resistive memories, augmenting them with peripheral circuits to facilitate logical, arithmetic, and multiplication operations. These memories are attractive because of their lower leakage and higher density compared to SRAMs and DRAMs.

1) MEMRISTIVE MEMORIES

A non-volatile memristive cell stores data via its conductivity. In single-bit storage, binary 0 and 1 correspond to high resistance state (HRS) and low resistance state (LRS), respectively. Memristive storage technologies include Resistive RAM (ReRAM), Spin-transfer Torque RAM (STT-RAM), and Phase Change Memory (PCM) [8], with ReRAM being widely used. It consists of a TiO2 or HfO2 layer between electrodes, supporting multi-bit storage. As shown in Figure 2(a), ReRAM arrays adopt a crossbar structure with memristive cells at row-column intersections. Writing occurs by applying voltage pulses to bitlines and wordlines, altering the middle layer's resistance, while reading is done by activating a wordline and measuring current or voltage through bitlines. Additionally, memristive memories enable in-memory arithmetic and logic operations via peripheral circuitry for analog computation.

2) CiM-BASED VECTOR MATRIX MULTPLICATION

Figure 2(a) shows a 4×4 crossbar where a 4×4 weight matrix is encoded in the cells' conductance (G). A 1×4 input vector can be applied to the crossbar in the form of analog voltages via Digital-to-Analog Converters (DACs). The resulting current in each memristor cell is the product of the input voltage and the cell's conductance. Summing the currents from the cells along a column, as detected on the bitline, completes the dot product operation. To capture the results of these computations, Analog-to-Digital Converters (ADCs) are essential. Weight and activation slicing is performed to accommodate the limited bit-storage capacity of cells and to sequentially apply activation bits to the crossbar over multiple cycles due to the bit precision constraints of ADCs and DACs.

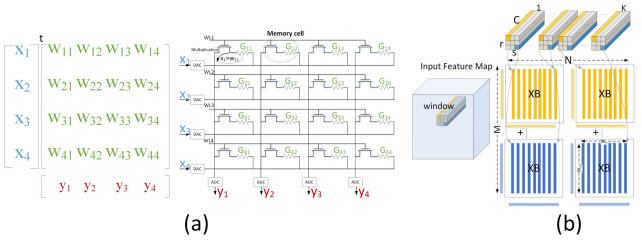


FIGURE 2. (a) Memristor crossbar and mapping of a CiM-based vector matrix multiplication of the crossbar (b) Conventional weight mapping for convolution operation.

Additionally, in the analog CiM approach, the non-idealities of crossbar, ADCs, DACs, and memristor variations limit the reliability of the hardware, which must be accounted for. In contrast, a digital CiM approach activates only a single row, avoiding the use of ADCs, but sacrifices maximum parallelism by not activating all rows [22] and [23].

3) WEIGHT MAPPING FOR CONVOLUTIONAL LAYERS

Convolutional layers in CiM platforms require unrolling filters into a weight matrix for crossbar mapping. Figure 2(b) illustrates weight mapping, where (K) filters of $(r \times s \times C)$ are rearranged into an $(rsC \times K)$ matrix. If this exceeds the crossbar's $(W_m \times W_n)$ capacity, multiple crossbars $M \times N = \frac{r\times s\times C}{W_m} \times \frac{K}{W_n}$. During inference, IFM windows matching filter size are unrolled and fed into crossbars, producing partial OFM vectors, which are aggregated into complete OFM elements. Fully connected layers can be viewed as convolution with 1D IFM and filters.

III. PREVIOUS WORKS

CiM platforms encounter various physical and circuit challenges, with research efforts addressing reliability and precision in crossbar memristors. Other studies focus on architectural aspects, integrating memristor crossbars as vector matrix multiplication units. Since this paper aims to develop a CiM-based architecture, the next subsections will explore existing research on CiM architectures and weight mapping methods.

A. CiM ARCHITECTURES

The ISAAC architecture emerges as a pioneering design, featuring a three-level structure comprising the chip, tile, and IMA, specifically designed for CNNs [5]. Data transfer

between tiles is facilitated by a NOC, while a shared bus system is employed within the IMA and tiles for input distribution and the aggregation of partial outputs from CiM units. Building upon the foundation laid by ISAAC, several architectures have been introduced in recent years. For example, ReHy mirrors ISAAC's hierarchical structure but also integrates the training phase into its design [9]. Additionally, another work presents a tile-based ISAAC-like array utilizing a NOC for data transfer and introduces a diagonal weight mapping strategy to optimize data reuse [11]. Furthermore, FullReuse incorporates a unique internal communication channel within each tile to enable data reuse through CiM units, where filter weights are replicated to enhance efficiency [10]. The Domino architecture consists of an array of directly interconnected tiles that integrates both computational and data transfer mechanisms within CiM units using the on-the-move computation technique. FORMS presents a CNN accelerator that prunes the weight matrix to reduce its size, utilizing a shared communication bus for data transfer between ReRAM arrays [12]. FloatPIM in [33] presents a digital CiM approach for implementing floating-point MAC operations using NOR logic within the rows of a memory block. Each layer is mapped to a separate memory block, and data transfer is managed through direct switches for cascading layers. Reference [34] partitions the memory into morphable and storage blocks, designated for training and inference, respectively. The design consists of 19×2 ReRAM subarrays and establishes direct connections between these blocks. The aforementioned studies predominantly employ a shared-bus architecture at various levels of their design to facilitate access to CiM units.

A few studies have introduced CiM-based architectures described as systolic. While many works have employed Von Neumann-style systolic arrays like [36], this review focuses exclusively on CiM-based systolic architectures. Among those utilizing ReRAM arrays, [26] employs a 1D systolic array inspired by FIR filters for CNNs, which necessitates conventional weight mapping. In [27] and [28], the analog computation within ReRAM arrays is referred to as systolic, with efforts to provide more precise circuit-level models for ReRAM-based processing of Gaussian filters. Similarly, [29] implements RNNs by formulating the Fourier transform and point-wise operations as in-situ MAC operations mapped to ReRAM crossbars, incorporating a systolic dataflow for communication between crossbar cells. Other approaches focus on structured decomposition and mapping techniques. For instance, [30] decomposes large dense matrices, mapping each sparse decomposed matrix to a memristor crossbar using a path-based method while arranging crossbars in a array without defining an specific dataflow between crossbars for CNNs.

In digital SRAM-based architectures, [20] and [21] employ a 1D systolic array inspired by FIR filters. Reference [22] optimizes convolution operations by transferring input data between SRAM cells, thereby reducing the cost associated with adder trees used for accumulating partial sums. To further enhance computational parallelism, [23] maps weight parameter bits to different SRAM banks, organizing SRAM banks into a systolic array for vector-matrix multiplications. Additionally, [24] introduces a 2×2 array of SRAM banks interconnected vertically, horizontally, and diagonally to efficiently manage input, weight, and output-stationary dataflows for CNNs. Furthermore, [25] proposes a hybrid system that integrates digital SRAM-based CiM with von Neumann systolic hardware for both forward and backward operations in CNN training.

Existing works on CiM- and ReRAM-based systolic architectures for CNNs are limited in scope, often focusing on 1D systolic dataflows [26], analog inference within ReRAM [22], [27], [28], [29], or bit-level computations rather than full CNN layer processing [23]. Furthermore, only a few works, such as [20] and [21] which are 1D systolic, have evaluated their architectures on CNNs with model parameters at the scale of VGG16, primarily using the MNIST dataset, which is significantly smaller than ImageNet. This work introduces a 2D deep systolic dataflow across multiple hardware levels, incorporating weight mapping and pruning strategies for DCNNs. It also presents sparse SCiMA, the first systolic CiM-based hardware to exploit sparsity. Furthermore, a system-level framework is proposed, encompassing application compilation, real-world workload evaluation, and a comparative analysis of analog computation.

B. WEIGHT MAPPING STRATEGIES

Various studies have aimed to enhance input data reuse in convolutional layers by duplicating weights within the memristor crossbar. Reference [5] improves computational efficiency by distributing identical weight matrices across multiple crossbars, enabling parallel input batch processing. Reference [11] arranges filter weights diagonally to prevent interference and facilitate parallel computation, though it underutilizes non-diagonal memristors and increases area overhead. Similarly, FullReuse [10] and AtomLayer [14] replicate weights across crossbars and introduce specialized communication channels for broadcasting input data to processing elements. Max2 enhances parallelism by replicating weight mapping across multiple crossbars [15]. The Shift and Duplicate Kernel (SDK) technique reuses input data and weights by shifting and duplicating kernels onto adjacent columns, creating a 'parallel window' for computing multiple OFM elements in a single cycle [16]. STICKER-IM enables the activation of 32 rows of SRAM to maximize parallelism. Since the number of channels in the initial layers is fewer than 32, it maps the first layers using a kernel-channel order, while the later layers follow the conventional channel-order mapping [31]. CiMloop [32] provides a framework for simulating CiM-based memory blocks under different digital and analog computation schemes, exploring various memory cell configurations.

This paper introduces two CM-IntraCWM and KM-InterCWM strategies for mapping weight to memory crossbars aimed at decreasing data transfer and augmenting data

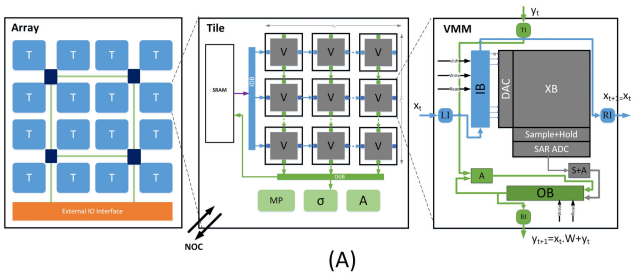


FIGURE 3. Hierarchical SCiMA architecture: VMM: basic vector matrix multiplication unit, Tile: a VMM array with a systolic dataflow, Array: an array of Tiles interconnected through NOC.

reuse of CNNs without duplication of weight parameters onto memristor crossbar. In addition, structured pruning is applied to dense layers to reduce the area overhead of ReRAM without incurring the computational complexity associated with sparse matrices.

IV. PROPOSED ARCHITECTURE

The first subsection explains proposed SCiMA for implementation of DCNNs. In the second subsection, the SCiMA dataflow and pipelined design would be presented.

A. SCiMA ARCHITECTURE

The architectural layout is depicted in Figure 3, where SCiMA is composed of three hierarchical levels: Array, Tile, VMM explained in the following subsections.

1) VMM

VMM, as the most basic unit of SCiMA, performs analog multiplication of an input vector against a weight matrix preserved within a memristor crossbar, yielding an output vector. This memristor crossbar, of size S, accommodates a weight matrix sized at $(M_v \times N_v) = (S \times \frac{S \cdot m}{w})$, with m and WW representing the storage capacity per cell and bitwidth of weights. Consequently, the multiplication process involves an input vector by size M_v, producing an output vector of size N_v. The input and output vectors are housed within input and output buffers (IB and OB) respectively. For the analog computation, DAC components are used to apply the input vector onto the crossbar via an input streaming mechanism, while Successive Approximation Register (SAR)-ADCs are tasked with transforming the bitline's output voltage into digital form. The Sample and Hold (S+H) units are responsible for maintaining the bitline's analog output voltage. Ultimately, the Shift-Add (S+A) function aggregates the weighted partial outputs of bitlines, derived from the weight slicing and input streaming processes.

The VMM's functionality is illustrated using three distinct colors: blue, green, and dark, to represent the three pipeline stages—namely, the Input Transfer Controller (ITC), Output Transfer Controller (OTC), and Computation Controller (CC), respectively. This pipeline stages facilitates the differentiation of the dataflow for inputs, outputs, and computations, which operate independently, thereby enabling a mechanism for self-timed adjustments. Initially, the ITC stage acquires the input vector x at time t (denoted as x_t) via the Left Interface (LI) and stores it into the IB. This input vector is subsequently relayed to the adjacent VMM at time $t+1$ (denoted as x_{t+1}) through the Right Interface (RI) to the lateral slave VMM. Upon activation by the ITC, the CC stage commences the analog computation of $x_t \cdot W$, with the end result being stored in the OB. The OTC stage then takes over, receiving the partial output vector y_t from the top master VMM. It combines this with its own computed partial output and forwards the aggregated partial output vector to the bottom slave VMM at y_{t+1}. A straightforward burst request-acknowledge handshaking protocol is employed for interfacing between VMMs. To support negative weights, the weights are added with a bias. The Tile calculates the sum of the input vector's data and sends it along with the input vector to the VMM, where the multiplication of the sum and bias is subtracted from the VMM result. To facilitate a systolic data flow, an array of VMMs is interconnected through pinning interfaces on all four sides. This configuration generates two rhythmic streams of input and output data—one flowing input data from left to right and the output data from top to bottom respectively.

TABLE 1. Tile configuration paraemters.

	Parameters
Computation Configuration(CC)	H, W, C, K, Stride, r, s, MP, Act
Input Configuration (IC)	$D^i : (H^i_1, H^i_2)(W^i_1, W^i_2)(C^i_1, C^i_2) \mapsto TileID^i$
Output Configuration (OC)	$D'^i : (H'^i_1, H'^i_2)(W'^i_1, W'^i_2)(C'^i_1, C'^i_2) \mapsto TileID'^i$
Partial result Configuration (PC)	$D'''^i : (H'''^i_1, H'''^i_2)(W'''^i_1, W'''^i_2)(C'''^i_1, C'''^i_2) \mapsto TileID'''^i$

2) TILE

A Tile consists of an $(m_v \times n_v)$ array of VMMs, interconnected through interfaces on all four sides, forming a weight matrix of dimensions $(m_v \cdot M_v \times n_v \cdot N_v)$. In scenarios where not all VMMs of a Tile are utilized, the VMMs can be activated or deactivated via interface flags. Dual-port SRAM is utilized for storing input and output data, in conjunction with two buses—IDB for distributing input data to lateral VMMs and OGB for collecting outputs from the bottom VMMs. Additionally, the Tile incorporates max pooling (MP) and activation function (σ) units to perform post-processing on the matrix multiplication layers of the CNN. The adder A is responsible for combining the partial sums of individual Tiles with the aggregated partial sums of Tiles that contribute to the final result.

To accommodate various sizes and types of CNNs, Tiles are programmable through application compilation and are linked to a NOC. The Tile configuration is divided into four sections, as indicated in Table 1. The computation parameters include the dimensions of the input data (H, W, C) to be processed, the kernel dimensions (r, s, K, stride), and flags for activating pooling and activation functions if the Tile holds the final result of a CNN layer. Notably, parameters for fully

connected layers are adaptable to those of convolution layers. Data transfer between Tiles is based on Input, Output, and Partial result configurations (represented as IC, OC, and PC), comprising a list of tile IDs along with the dimensions of the 3D data segments, as shown in Table 1. D, D', and D'' are the dimensions of the input, output, and partial sums data, respectively. *i* denotes the i'th element of the configuration.

Three pipelined input flow, output flow, and computation flow controllers (IFC, OFC, CFC) manage the data transfers and computations. The IFC awaits the completion of all input partitions from the Tiles of the preceding layer, as listed in IC. CFC segments and vectorizes the inputs based on the weight mapping strategy and CC parameters, dispatching them to the corresponding left column of VMMs connected to the IDB. The flow of input vectors is colored blue. In parallel, CFC aggregates downstream partial sums (colored green) from VMMs, including the final partial sums via OGB. Once all output values are collected, OFC waits to receive the partial sum from the tiles listed in PC. Upon integrating the partial sum with its own results, the Tile dispatches the batch of output values to the destination Tile as defined in OC. The CFC comprises counters initialized with CC parameters of convolution to index IFM windows and filters, facilitating the generation of data addresses stored in SRAM for communication with VMMs. It also includes flags to enable the activation function in the final result path and the max pooling unit. Additionally, the IFC and OFC incorporate tables containing IC, OC, and PC parameters to monitor and verify the completion of data transfers upon packet reception, along with counters to store data at predefined SRAM addresses.

3) ARRAY

At the Array level, Tiles are interconnected via a NOC, enabling data communication within and between the Tiles of different application layers. To minimize the area overhead associated with the interconnect, a concentrated-mesh (C-mesh) NOC architecture is employed. This design allows for the connection of four Tiles (nodes) to a single router, effectively reducing the number of routers by a quarter. The mapping algorithm ensures that Tiles within the same layer and those across different layers are mapped to adjacent Tiles. This strategy primarily facilitates data transfers between neighboring Tiles and justifies the C-mesh *NOC*.

B. SCiMA DATAFLOW

Figure 4 illustrates the dataflow within a multilayer CNN. As shown in Figure 4(a), each CNN layer is mapped to a Tile subarray. The weight parameters are expanded into a matrix of dimensions $(M_w \times N_w)$, and the tTle subarray is determined based on the formula $\left(\frac{W_x}{m_v \cdot S} \times \frac{W_y}{n_v \times \frac{S \cdot m}{W}}\right)$. If the weight matrix dimensions do not precisely match the Tile subarray, some VMMs remain unused. The architecture incorporates three distinct data flows, managed via the NOC:

- **The intra-layer input flow**, which involves the broadcast of input data to Tiles within the same row.
- **The intra-layer partial result flow**, which aggregates the outputs from Tiles within the same column of an specific layer.
- **The inter-layer data flow**, transmits the output from Tiles holding the final results to facilitate their broadcast across the Tile rows of the next layer.

Figure 4(b) depicts a pipelined design for Tile-based layers. Due to limited Tile buffer capacity, large input data is divided into sub-batches processed sequentially in a pipeline. Each batch from the previous layer's Tiles propagates across a corresponding row in the next layer, allowing multiple batches to be processed simultaneously. The bottom-layer Tiles, holding final outputs, forward output sub-batches to the next layer. The pipelined architecture engenders a systolic dataflow, enabling the rhythmic transfer of inputs and partial sums both vertically and horizontally at the Array level.

Within the Tile, as shown in Figure 4(c), the input controller selects a specific window of the IFM and initiates its unrolling in accordance with the weight mapping to generate input vectors. These vectors are then divided into sub-vectors, which are dispatched to the respective rows of VMMs. The figure shows the IFM window and the associated sub-vectors by utilizing identical colors. Furthermore, the rows of VMMs that store the weight matrix elements corresponding to the sub-vectors are marked with analogous grid patterns.

The sub-vectors are sent to the VMMs by channeling them into the corresponding lateral VMMs. Upon reaching the VMMs, the sub-vectors undergo processing. The partial sums generated by the VMMs migrate vertically from top to bottom. This orchestration culminates in a systolic dataflow, wherein each VMM represents the intersection of two concurrent waves of input and output data flows. Diagonal VMMs, color-coded accordingly, represent the computational wave's synchronized peaks and troughs. Since analog computations within ReRAM are characterized by a systolic computation paradigm, SCiMA maintains a deep systolic flow across multiple architectural levels (Array, Tile, and VMM) to enable an efficient systolic dataflow.

V. PROPOSED WEIGHT MAPPING FOR EFFICIENT DATA TRANSFER

Convolutional layers are among the most feature-dominant layers and represent the primary bottleneck in inference latency. An optimized weight mapping strategy is proposed to enhance data movement and reuse of feature data, as discussed in Sections V-A and V-B. For dense layers, which contribute the most to the hardware's area overhead, structured pruning is employed to reduce the number of weight parameters, thereby minimizing resource demands.

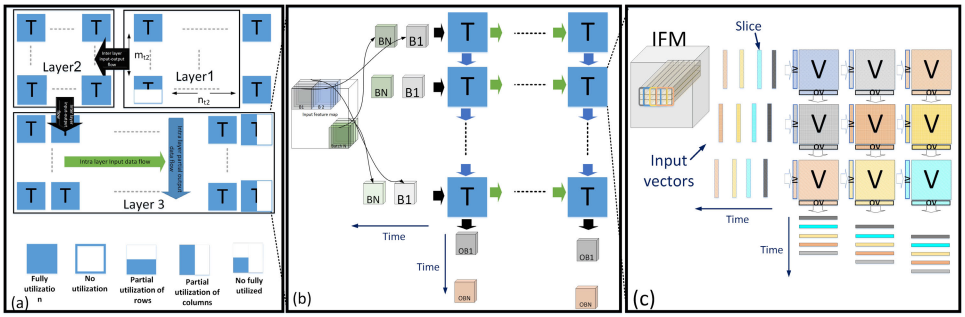


FIGURE 4. (a) Mapping different matrix multiplication layers of CNN to a subarray of Tiles (hardware layer) (b) Dataflow of batch data for a convolutional layer inside hardware layer (b) The dataflow of input vectors generated by slicing and unrolling consecutive input windows within a Tile.

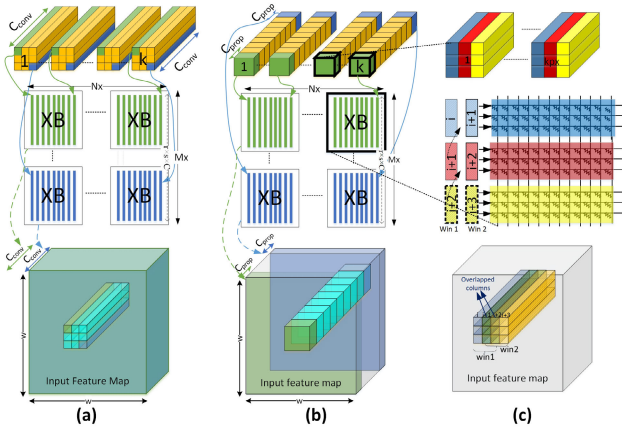


FIGURE 5. Conventional weight mapping in channel-major order; (b) Proposed KM-InterCWM weight mapping in kernel-major order; (c) Proposed CM-IntraCWM weight mapping in column-major order.

A. KERNEL MAJOR INTER CROSSBAR WEIGHT MAPPING (KM-INTERCWM)

KM-InterCWM aims to enhance input data exclusivity by reducing redundancy in input values distributed across the Tiles. It involves segmenting filters and arranging them into a crossbar column, thereby encompassing the fewest possible input data channels. Since the filter slices traverse the X-Y plane, covering all elements of the IFM corresponding to the kernels of filter slice, the number of kernels allocated to a crossbar directly influences the required data transfer of IFM for both the crossbar and its principal Tile.

Figures 5(a) and (b) compare the conventional and proposed methods for mapping weight of K filters each with C kernels. In the conventional approach, filters are unrolled in a channel-major fashion, mapping the maximum number of input channels to a crossbar, C_{conv} equivalent to crossbar size S. In contrast, the proposed weight mapping unrolls filters in a kernel-major manner, resulting in $C_{prop} = \frac{S}{r \times s}$ channels mapped to each crossbar. Consequently, the volume of input data transferred to each crossbar is $(H \times W \times S)$ in the conventional method and $\frac{(H \times W \times S)}{r \times s}$ in the proposed method. Thus, the data transfer of input data to Tiles is reduced by a factor of $r \times s$ which is kernel size.

B. COLUMN MAJOR INTRA CROSSBAR WEIGHT MAPPING (CM-INTRACWM)

CM-IntraCWM strategy is employed to arrange the subfilter weights determined by KM-InterCWM within the crossbar. This arrangement is optimized to maximize the reuse of data from overlapping, consecutive windows, thereby reducing data transfer requirements during the sliding window operation. With CM-IntraCWM, the architecture ensures that the most frequently accessed data remains close to the computational units, minimizing latency and energy consumption associated with data movement.

Figure 5(c) illustrates the convolution process, which involves scanning two successive windows of the IFM using 3×3 filters with a stride of 1. The initial window encompasses columns i to $i + 2$, while the subsequent window spans columns $i + 1$ to $i + 3$. Consequently, two-thirds of the data between these windows overlap, and only one-third of the data from the previous window is replaced with new data for the next window. Essentially, as the window slides, columns $i + 1$ and $i + 2$ shift to columns i and $i + 1$ of the previous window, while column $i + 3$ is introduced as the new column. Accordingly, in CM-IntraCWM, the weights of the subfilters are organized within the crossbar in a column-major sequence, as shown in the figure. The columns of subfilters, from right to left, are mapped onto the rows of the crossbar, from top to bottom. For $r \times s$ filters, data reuse is maximized by $\frac{s-Stride}{s} \times 100\%$.

C. SPARSE SCiMA: STRUCTURED PRUNING FOR DENSE LAYERS

Dense layers perform vector-matrix multiplications, leading to lower inference latency compared to the spatially complex computations in convolutional layers (CNNs). However, they constitute a substantial portion of the weight parameters in DCNNs. As depicted in Figure 6(a), dense layers can account for up to 90% of the total weight parameters in DCNNs on average. Given that ReRAM-based CiM architectures employ a weight-stationary paradigm, dense layers significantly contribute to hardware overhead. To address this issue, pruning techniques are employed to eliminate redundant or less significant weights, thereby reducing model size, inference

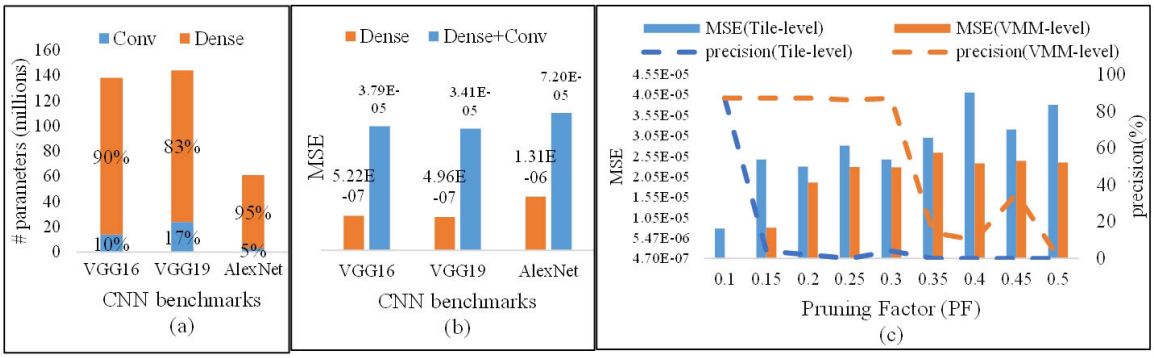


FIGURE 6. (a) Number of parameters of dense and convolution layers for DCNN benchmarks, (b) MSE for pruning dense and dense+conv layers, (c) Impact of sparsity on accuracy at Tile and VMM levels in VGG16.

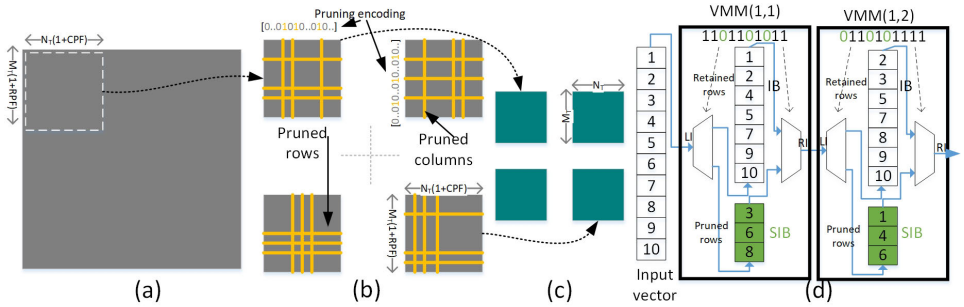


FIGURE 7. (a) Main weight matrix of dense layer (b) pruning each sub matrix (c) pruned sub matrices mapped to VMM (d) sparse VMM handling sparsity.

latency, and energy consumption. While fine-grained sparsity offers advantages in digital architectures, it is less effective in CiM due to challenges associated with handling sparse data and managing the overhead of nonzero elements [31].

Structured pruning effectively preserves the network topology while reducing model size, enabling execution on the same hardware with minimal area overhead. Among those works exploiting sparsity, Bit-level sparsity [40], [41] maps individual weight bits to different crossbars, pruning those crossbars with many zero bits. Column-wise sparsity, as proposed in [42], identifies and removes columns of the weight matrix that have minimal impact on the network's mean squared error (MSE). Approaches such as [43] focus on identifying submatrices with zero values, though they are constrained by the limited number of activated rows and columns per analog inference cycle. The methods in [44], [45] identify recurring patterns and structured shapes within sparse matrices, unifying all submatrices into a set of patterns. This approach reduces the overhead of sparsity exploitation by limiting the number of sparsity patterns. Additionally, [46] introduces block-based pruning by eliminating rows and columns within matrix blocks and mapping them onto crossbars. It involves retraining to minimize the impact of pruning on the performance. However, it does not define explicit criteria for selecting pruning factors or provide detailed hardware implementation insights.

To enhance weight storage efficiency and minimize area overhead, this work integrates structured pruning within a systolic CiM-based architecture. The proposed approach maintains a balance between accuracy degradation and hardware constraints, enabling efficient execution. Notably, although this hardware is designed for our structured pruning approach, it can also be utilized for future efficient pruning methods.

As depicted in Figure 7, the weight matrix of a dense layer (Figure 7(a)) is partitioned into sub-matrices (Figure 7(b)). The pruning algorithm removes a fraction RPF of rows and CPF of columns with the lowest L_2-norms, where RPF and CPF represent the Row and Column Pruning Factors, respectively. This method operates on pre-trained CNN models without additional training. The retained weight blocks form a compact weight matrix of size $M_T \times N_T$, which aligns with the architectural storage constraints, as shown in Figure 7(b).

To analyze the impact of pruning on dense and convolutional layers, Figure 6(b) presents the effect of pruning on the MSE for DCNNs, evaluated on 10,000 instances of the ImageNet dataset. The comparison considers two scenarios: pruning applied exclusively to dense layers and pruning applied to both dense and convolutional layers. Simulation results with a pruning factor (PF) of 0.3 for both RPF and CPF reveal that pruning convolutional layers significantly increases the MSE of VGG16 and VGG19 to the 10^{-5} range. In contrast, restricting pruning to dense layers maintains MSE within the 10^{-7} range, thereby minimizing accuracy degradation while optimizing hardware area utilization.

TABLE 2. Comparison of SCiMA and other frameworks.

Framework	Language (Python+)	Performance	NOC	Power	ReRAM Modeling	Compiler	Sparsity	Train	Circuit level
SCiMA	SystemC	RW	Yes	J/OP	M	Yes	Yes	No	low
NeuroSim [19]	C++	A	No	DD	For loop	No	No	Yes	high
TimeLoop [37]	C++	No RW	Yes	J/OP	N/A	No	No	No	medium
CiMLoop [32]	C++	No RW	Yes	J/OP	For loop	No	No	No	medium
MNSIM [39]	MATLAB & C++	A	No	DD	For loop	Yes	No	Yes	medium
A: Analytical, RW: Real Workload, DD: Data-value-Dependent, M:Matrix-based									

To determine the appropriate granularity of pruning at the Tile or VMM level and the optimal pruning factor (PF), Figure 6(c) presents the average MSE and accuracy of VGG16 networks for different PF values. This work adopts a uniform PF for the model layers. As observed, Tile-level pruning results in higher MSE compared to VMM-level pruning across various PF values. This discrepancy arises because finer granularity distributes the induced error more evenly across the model. Moreover, accuracy of application declines significantly at PF $= 0.3$ for VMM-level pruning, whereas this drop occurs earlier at PF $= 0.1$ for Tile-level pruning. However, fine-grained pruning necessitates additional registers to manage sparsity efficiently. Considering this trade-off, we identify PF values of 0.33, 0.3, and 0.27 as optimal points that balance accuracy preservation and resource overhead for VGG19, VGG16, and AlexNet, respectively.

To ensure data integrity during data transfer between dense layers, the retained and pruned rows and columns are encoded as binary streams of 1s and 0s. These streams are then arranged within the VMMs as part of their configuration. Figure 7(d) illustrates the VMM enhancement designed for handling sparsity in pruned rows of submatrices mapped to a VMM. The Tile architecture assumes a VMM with an input vector size of $S \times (1+PF)$, which sends an input vector to the rows of the VMM. According to the pruning configuration, VMM(1,1) dictates the retention of non-pruned rows in the input buffer (IB), while pruned rows (3, 6, and 8) are temporarily stored in a shadow input buffer (SIB). The complete input vector is then transmitted to the adjacent VMM(1,2) by sending both the IB and SIB data in the appropriate order as directed by the pruning configuration. Subsequently, VMM(1,2) stores pruned rows 1, 4, and 6 in its SIB while performing computations on the retained rows stored in the IB.This approach incurs some area overhead due to the shadow buffers. A similar strategy must be employed to transmit the output data of the VMMs, ensuring that both retained and pruned columns are appropriately managed.

VI. VIRTUAL FRAMEWORK

This manuscript introduces a framework designed for the SCiMA, enabling hardware assessment in terms of performance, power, and computational reliability. The framework supports CNNs of various sizes and quantization levels while incorporating structural and back-annotated circuit-level

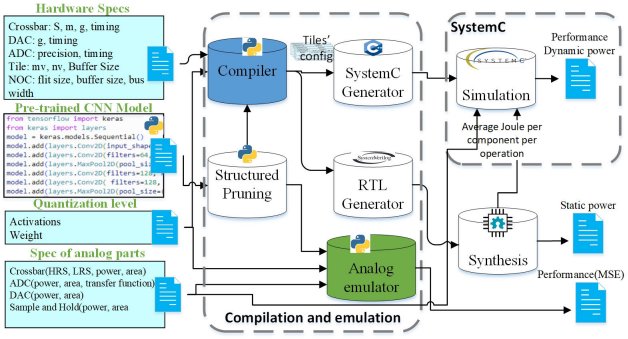


FIGURE 8. System-level framework for compilation and simulation.

information. Table 2 compares the SCiMA framework with other frameworks for modeling ReRAM-based architectures. Unlike existing frameworks, which primarily use C++ for estimating area, latency, and power in neuromorphic computing hardware, this work employs SystemC. SystemC bridges high-level modeling with hardware components, allowing for more precise and reliable simulations. Additionally, SCiMA uniquely supports real workload (RW) evaluations for performance analysis, whereas other works rely on analytical models or non-real workload estimations. SCiMA integrates a dedicated compiler to configure hardware and manage data transfers within the architecture. Its matrix-based approach to ReRAM modeling—operating at a macro level—ensures efficient execution on parallel computing hardware. Moreover, SCiMA explicitly addresses sparsity and pruning in CNNs, improving hardware utilization for processing sparse data. Like TimeLoop and CiMLoop, the SystemC model incorporates $\frac{joules}{operation}$ of components for power estimation. Finally, SCiMA has not yet delved into circuit-level details as extensively as some other frameworks.

As shown in Figure 8, the framework comprises two main components: Compilation-Emulation (CE) and SystemC modeling, highlighted in this work. The CE stage takes hardware specifications such as buffers, crossbar, ADC, DAC, VMM, and router configurations along with Keras-defined CNN models, including quantization parameters like activation and weight width denoted as AW@WW. First, the quantized pre-trained CNN undergoes pruning using the proposed method in Section V-C, extracting indices of pruned and retained rows and columns. The compiler then processes the pruned model to extract Tile configurations. The compiler's

output feeds both the *SystemCgenerator* and *RTLgenerator*, producing the SystemC model and RTL hardware description. The pruned CNN model, combined with circuit characteristics of analog components impacting performance, is provided to the analog Emulator to evaluate computational error (see Section VI-A3). The synthesized RTL design, excluding analog components, is performed to extract power consumption and area per component. For precise dynamic power estimation, the SystemC model integrates back-annotated circuit-level information of components' energy. The impact of analog components is accounted for by aggregating their power and area overheads. Additionally, hardware timing constraints, such as back-annotated transfer rates, clock frequencies, and architectural timing details, are incorporated into the SystemC model.

A. COMPILATION-EMULATION (CE) ALGORITHM

1) OVERALL VIEW

The algorithm, illustrated in Algorithm 1, is color-coded in blue and green to distinguish compiler and emulator components, respectively, which are explained in the following subsections. This work employs an object-oriented Python program for compiling applications and performing analog hardware emulation. The structured approach allows easy modifications by porting methods to accommodate different analog models and compilers for various applications. For example, additional architectural levels can be integrated into the existing framework, and corresponding compiler passes can be defined to handle each level's unique characteristics—such as memory hierarchies, dataflows, or crossbar configurations—without requiring major changes to the overall codebase. As shown in Figure 1, three primary classes **VMM, Tile**, and **Architecture** are defined in lines 1, 6, and 28, corresponding to each hardware level. Additionally, an intermediate hardware layer class (**HL** in line 16) is defined for Tiles of an application layer in order to simplify compilation. Each class includes a constructor defining its specific hardware features. For example, the **VMM** class constructor initializes parameters like crossbar architecture, ADC, DAC, and buffer properties. To enhance portability and streamline modeling, a hierarchical compilation and modeling approach is employed. Each class contains a **mapping** method that assigns algorithm portions to corresponding lower-level components, ensuring modularity and ease of hardware design modification.

Algorithm 1 Compilation-Emulation Algorithm

```
1:  class VMM:
2:      constructor(VMMfeatures);
3:      G, WM=mapp(Part_v, CM-IntraCWM);
4:      AnalogInference(InputVector)*;
5:  ---------------------------------------------------------
6:  class Tile:
7:      constructor(VMMfeatures, TileFeatures);
8:      mapp(Part_t):
9:          New VMM[m_v, n_v].mapp(Part_v)=(Part_t,KM-InterCWM)
10:     configTiles(Part_t):
11:         self.setCC(r, s, C, K, H, W, MP, act)
12:     inference(IFM):
13:         IVs=input-distribure-collector (IFM, KM-InterCWM);
14:         VMM.inference (IV);
15: ---------------------------------------------------------
16: class HL:
17:     constructor(VMMfeatures, TileFeatures);
18:     mapp(Layer):
19:       New Tiles[m_t, n_t].mapp(Part_t)=partitioning(Layer, KM-InterCWM);
20:     configTiles():
21:       For each Tile[i, j] except i=m_t
22:           Tiles[i, j].setOC(Tiles[i, j].D', Tiles[i + 1, j])
23:           Tiles[i + 1, j].setPC(Tiles[i, j].D', Tiles[i, j])
24:     inference(IFM):
25:         IFM_ps=input-distribure-collector (IFM, KM-InterCWM);
26:         Tiles.inference (IFM_p);
27: ---------------------------------------------------------
28: class Architecture:
29:     constructor(VMMfeatures, TileFeatures, NOCfeatures);
30:     mapp(Application);
31:       For each MM layer of Application: New HL(MMlayer);
32:     configTiles():
33:       For each HL_i;
34:         r ← 0
35:         For each tile of last row Tiles[i, m_t]
36:             OD_remained ← HL_i.Tiles[i, m_t].D'
37:             ID_remained ← HL_{i+1}.Tiles[r, 0].D
38:             While the whole OD_remained is not set
39:                 PartialD ← MAXoverlap(ID_rem, OD_rem)
40:                 HL_i.Tiles[i, m_t].setOC(PartialD, HL_{i+1}.Tiles[r, :])
41:                 HL_{i+1}.Tiles[r, :].setIC(PartialD, HL_i.Tiles[i, m_t])
42:                 OD_remained.drop(PartialD), ID_remained.drop(PD)
43:                 IF the whole ID_remained is mapped
44:                     r ← r + 1, ID_rem ← HL_{i+1}.Tiles[r, 0].D
45:   inference(input):
46:       HL.inference();
47: ---------------------------------------------------------
Require:
48: VMMfeatures: HRS, LRS, ErrorDistribution(ED), S, m, p, g
49: TileFeatures: m_v, n_v, TileMem
50: NOCfeatures: flitSize, packetSize, bufferSize
51: Application: CNN, AW@WW
52: Weight Mapping: WM : CM − IntraCWM, KM − InterCWM
Ensure:
53: TileConfigurations: id, CC, PC, IC, OC
54: HDL codes: SystemVerilog Code, SystemC code
55: Application perofrmance: MSE
56: Function Main()
57:       QCNN=Quantize(CNN);
58:       prunedCNN=pruning(QCNN);
59:       ArchObj=New Architecture().mapp(CNN, WM)
60:       TileConfigs=ArchObj.ConfigTiles()
61:       placement(TileConfigs);
62:       generateHDLcode(TileConfigs);
63:       generateSystemCcode(TileConfigs);
64:       performance=MSE(ArchObj.inference(input), CNN.call(input))
```

2) COMPILER

In the **Architecture** class, the **mapp** method assigns an object of class **HL** to each matrix multiplication layer of the CNN and its subsequent activation or pooling layers (line 31). Within the **mapp** method of the **HL** class, layer weights are partitioned into partitions ($part_t$) based on KM-InterCWM and mapped to a Tile array, denoted as $[m_t, n_t]$. Further refinement occurs in the **mapp** method of the **Tile** class, where weight partitions ($part_v$) are mapped to a VMM array, denoted as $[m_v, n_v]$. Within the **mapp** method of the **VMM** class, $part_v$s are unrolled based on CM-IntraCWM, generating a weight matrix (WM) and a conductance matrix (G) using HRS, LRS, m, and error distribution (ED).

Following this, the **configTile** method is executed at each architectural level to set configuration parameters, leveraging internal information specific to each level. In the **configTile** method of the **Tile** class, computation configuration (CC)

parameters are assigned according to $part_t$ slice of CNN layer (lines 10-11). For data transfer configuration, the last row of Tiles, holding the layer's final result, broadcasts its output to the Tile rows in the next layer. This defines the input configuration (IC) of all Tiles and the output configuration (OC) of the last Tile row, managed in the **configTile** method of the **Architecture** class (lines 32-44). Meanwhile, all but the last row of Tiles in each layer send their output as a partial sum to the Tile below in the same column, which is handled in the **configTile** method of the **HL** class (lines 20-23), governing intra-layer partial sum data flow. For RESNET CNNs, modifications have been made to the compiler to handle skip inputs in residual block layers.

3) EMULATOR

As shown in Figure 1, each class of hardware levels includes an **inference** method that distributes data to lower-level components using the **input-distribute-collector()** method and then calls the **inference** method of lower-level components. This process manages digital accumulation and partial sum computations. At the lowest level, the **VMM** class employs the **AnalogInference()** method (line 4), which emulates analog computation within the memristor crossbar. This method takes an input vector and produces an output vector.

Each neuron's computation is mapped to a single column of the memristor crossbar and follows the formulation in Equation (1), as introduced in [38]. x and w denote activation and weight vectors, while v_k and G_k represent corresponding analog input and conductance values. The total conductance of a column associated with a given neuron is denoted as G_T:

$$y = \sum_{k=1}^{S} x_k \times w_k = adc(\frac{\sum_{k=1}^{S} v_k \times G_k}{\sum_{l=1}^{S} G_l}) = adc(\sum_{k=1}^{S} v_k \times \frac{G_k}{G_T}) \quad (1)$$

When weight and activation slicing, each weight is stored in n_c memristor cell storing m bits, and input is streamed in n_s g-bit slices. The computation reformulates as Equation (2) for $AW@WW$ configuration.

$$y = \sum_{i=1}^{n_s=AW/g} \sum_{j=1}^{n_c=WW/m} (2^g)^i \times (2^m)^j adc(\sum_{k=1}^{S} x_{k,i} \frac{G_{k,j}}{G_{T_j}}) \quad (2)$$

Since benchmark CNNs in this study contain millions of parameters, large crossbar arrays are required for efficient hardware emulation. To enhance simulation efficiency, the analog behavior of the crossbar is modeled using matrix multiplication, enabling parallel computation across the entire array. For this purpose, the input vector X is partitioned into g-bit slices, forming matrix X_s of dimension $S \times n_s$. An ideal DAC is assumed, but its non-ideal behavior can be leveraged for enhanced emulation. The analog matrix X_s is multiplied by G', where each element $G'_{i,j}$ is defined as

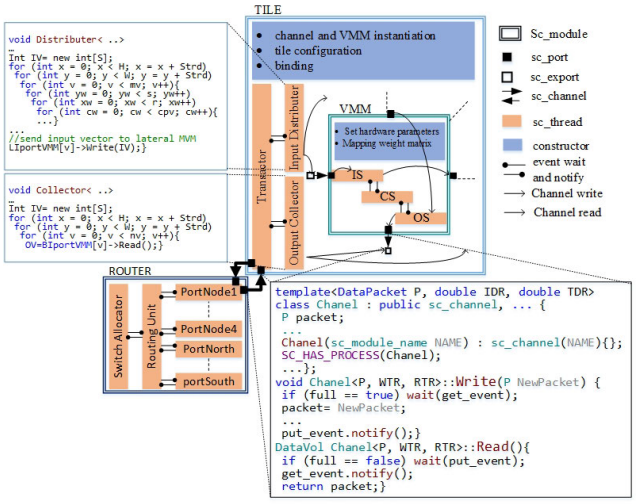


FIGURE 9. Graphical representation of structured SystemC modeling of proposed SCiMA architecture.

$G_{i,j}/G_{T_j}$. Equation (3) shows how this matrix multiplication enables full analog inference in a single operation:

$$Y_{n_o} = A(adc(dac([X_s]^T_{S\times n_s}) \times [G']_{S\times S}) \circ [CM]) \quad (3)$$

ADC modeling accounts for integral nonlinearity (INL) and differential nonlinearity (DNL) as described in [18]. The Hadamard product is used for element-wise multiplication between the result matrix and CM, which represents coefficients linked to crossbar columns and input slices. An aggregation function A is then applied to every n_c columns of the result matrix to generate the output vector of dimension n_o.

Appropriate amplifying has been applied to the analog output of each crossbar to ensure the analog output aligns with the input range of the ADC. In addition, sufficient rescaling has been applied to the output of each layer to compensate for quantization effects.

4) MAIN FUNCTION

In the **main** function (lines 56-63), the process begins with quantizing and structured pruning of the model. Following this, an **Architecture** object is instantiated based on the specified hardware and application features. Next, the hardware layers (**HL**s) are sorted according to the Tile array size $[m_t, n_t]$ and mapped accordingly to ensure the most efficient use of the smallest possible Tile array for the CNN model. A heuristic algorithm is employed for this allocation. The compilation phase involves generating HDL and SystemC code to evaluate the power, area, and timing characteristics of the hardware. Finally, the **inference** function of the **Architecture** object is executed to emulate computations and assess the application's performance (line 63).

B. SystemC MODELING

Figure 9 visualizes the primary components, their interconnections, and the hierarchical operational flow of

TABLE 3. Specifications of architecture.

	SCiMA		Sparse SCiMA	
Component	area(mm^2)	Power(mw)	area(mm^2)	Power(mw)
	VMM		Sparse VMM	
IB+SIB(128+38 words)	0.0028	0.74	0.0037	0.87
OB+SOB(16+5 words)	0.00028	1.01	0.00036	1.05
S+A	0.0001	0.007	0.0001	0.007
Controller	0.000037	0.3	0.00004	0.3
ADC-SAR(8-bit precision, 1.2MS/sec)	0.0012	2	0.0012	2
DAC(2 bits)	0.000085	0.6	0.000085	0.6
S+H	0.000005	0.00125	0.000005	0.00125
Crossbar(128×128)	0.000025	0.3	0.000025	0.3
Total	0.0045	4.98	0.0051(12.2%)	5.13(4.07%)
	Tile		**Sparse Tile**	
VMM array (4×4)	0.072	79.7	0.082	82.2
Controller	0.0007	0.23	0.0007	0.23
Adder	0.00003	0.004	0.00003	0.004
Pooling	0.00024	0.07	0.00024	0.07
Activation	0.00001	0.006	0.00001	0.006
SRAM (16KB)	0.083	20.7	0.083	20.7
Total	0.156	100.65	0.166(5.6%)	103.538(3.2%)
Router (16 flits)	0.007	8.07	0.007	8.07
	SCiMA(64×64)		**Sparse SCiMA**(48×48)	
Total	647.7	420.8W	384.49(-40.6%)	243.9W(-42.4%)

SystemC modeling. Three key **sc_module** entities Tile, VMM, and Router each represented with double-bordered shapes. The **constructor** and **sc_thread**s of these modules are color-coded in blue and green, respectively. Each component features multiple **sc_port** instances, depicted as solid black squares, enabling data exchange through communication channels. During **sc_module** instantiation, structural, power, and timing attributes, along with weight parameters, are initialized as **constructor** parameters. Specifically, the Tile **constructor** retrieves configurations from a predefined file which is output of the compiler. The **constructor** then binds the VMM array ($m_v \times n_v$) through dedicated channels and partitions the weight parameters to establish the corresponding weight matrix, while also configuring VMM interface flags.

Timing considerations are integrated using **wait** statements, which account for the clock frequency and RTL component activity in each computation round. Dynamic power estimation is conducted by monitoring hardware activity within execution **sc_thread**s, where power consumption is calculated based on the pre-characterized energy per operation of each component. This methodology provides a more precise evaluation of power efficiency under real workload conditions. SystemC modeling calculates the average energy consumption for each component type per action.

The VMM **sc_module** incorporates three **sc_thread** instances functioning as pipeline stages of ITC, OTC, and CC. ITC and OTC threads communicate with their **sc_port**s using **read** and **write** functions, illustrated as thin arrows in Figure 9, ensuring a structured data flow within the VMM. The Router **sc_module** is equipped with eight **sc_thread**s to manage data exchanges with four Tiles and four adjacent routers in the concentrated-mesh NoC. These **sc_thread** oversee data packet transmission and reception via target and initiator ports. Additionally, two supplementary **sc_thread**s handle packet routing and port assignments, with timing constraints aligned to the NoC's clock frequency and flit size.

Both the Router and Tile **sc_module**s incorporate dual initiator and target **sc_port**s, linked via modular event-driven channels represented as thick arrows. These channels inherit from the **sc_channel** class and use templates to instantiate different channel types based on packet type P and the data transfer rates of initiators (IDR) and targets (TDR), as depicted beneath Figure 9. **read** and **write** functions have been implemented to be called through interfaced ports. A simple version of this channel is used for VMM-VMM and Tile-VMM communications.

The Tile **sc_module** features an **sc_thread** labeled Transactor to transact with the NOC, tasked with aggregating data packets comprising input or partial result data, as well as packaging output data for dispatching to the target Tile. Upon the completion of input data acquisition, the Input Distributer **sc_thread** is activated by the Transactor **sc_event** signal to commence the vectorization of input data, dispatching it to the respective VMMs via **sc_port**, which interfaces with the VMM's LI port, as showcased in the upper left quadrant of Figure 9. In parallel, the Output Collector initiates the aggregation of partial outcomes from the lower-tier VMMs for each computational window.

To validate the SystemC model, the timing and output of the modules have been traced for both RTL and SystemC. The timing of the SystemC model has been calibrated against RTL implementations by fine-tuning existing **wait** statements and adding additional ones as necessary to synchronize execution. This process mitigates timing mismatches caused by data handshaking delays and controller variations. During verification and calibration, computational phases—such as the core computing (CC) phase responsible for vector-matrix multiplication—are abstracted using dual-layered loops.

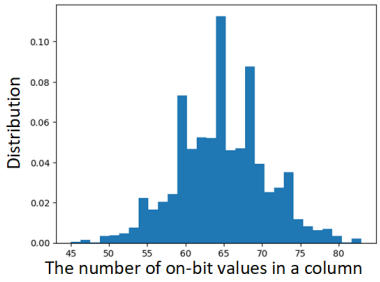


FIGURE 10. Distribution of number of one-bit values per column of crossbars for various CNN networks.

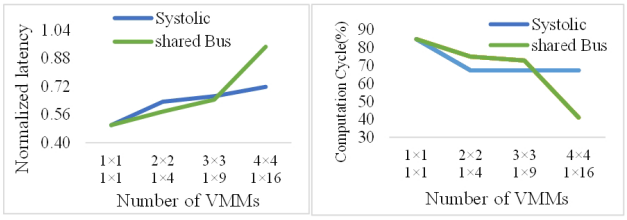


FIGURE 11. Scalability analysis: comparing computation cycle of each VMM and inference delay of convolution layer mapped to a Tile for different number of VMMs inside Tile configured in systolic and shared bus structure.

The SystemC-based architectural model is designed for modularity and configurability, allowing easy extension to support new architectural dimensions and custom bus structures. By updating specific **sc_thread** processes and communication channels, the system can be quickly adapted to represent different dataflows, memory hierarchies, or interconnect topologies.

VII. EXPERIMENTAL RESULTS

A. HARDWARE CHARACTERISTICS

The power and area specifications of SCiMA and sparse SCiMA in 32 nm technology are detailed in Table 3 for implementing CNNs with (16@8) quantization levels. The design employs a standard 128 × 128 ReRAM with a cell storage capacity of $m = 1$ [17]. Each VMM performs a multiplication between an input vector of 128 and a weight matrix of 128×16, yielding an output vector of 16. To support this, two buffers are required: one for input (128) and one for output (16). For activation processing, 16-bit values are applied in 8 groups of 2 bits each ($g = 2$) using a 2-bit DAC. To maximize crossbar efficiency, all rows are activated simultaneously. ADC precision is determined by analyzing the one-bit value distribution per crossbar column for all VGG layers (Figure 10). Since the one-bit count per column does not exceed 80, the maximum digitized column output is $m \times 80 \times (2^g - 1) = 240$, requiring an 8-bit ADC. Consequently, an 8-bit SAR-ADC is utilized as characterized in [18]. Each Tile comprises a 4 × 4 array of 16 VMMs and is equipped with a 16-KB SRAM buffer. Buffer size is minimized through a pipeline design. The NOC follows a concentrated mesh architecture with eight 16-flit buffers for data storage. The bitwidths of inter-VMM connections within a tile and inter-NoC interconnects are set to 16 bits and 32 bits, respectively.

The clock frequency for the digital components is conservatively set to 1 GHz, based on the critical path delay of the architecture. The area values are obtained through synthesis, while power consumption is estimated via SystemC simulations of the Tile and Router. This estimation incorporates the back-annotated $\frac{joule}{operation}$ of each component, extracted from synthesis. To achieve maximum activity, a convolutional layer with an input batch size of 16 × 16 was executed, utilizing all VMMs for weight storage. This approach provides a more precise power estimation compared to summing the peak activity power of individual components. To estimate the dynamic power of components, a switching activity factor of 0.2 has been assumed. As reported, SCiMA requires a 64 × 64 Tile array for implemetnation of DCNNs at the scale of VGG16 and VGG19. Sparse SCiMA reduce the configuration to 48 × 48 but introduce additional area and power overhead: sparse VMM and Tile consume 12.2% and 5.6% more area, respectively, and 4.07% and 3.2% more power. This increase is due to shadow input and output buffers (SIB and SOB) inside the VMM to handle pruned rows and columns. However, the overall sparse architecture reduces area and power consumption by 40.6% and 42.4%, respectively, demonstrating significant efficiency improvements.

B. EVALUATION OF SYSTOLIC DATAFLOW BENEFITS

To investigate the benefits of systolic dataflow, we compared the scalability of systolic arrays and shared bus structures used in state-of-the-art architectures evaluated in this paper. In the shared-bus architecture, both input and output data are transferred to the VMMs via a common bus, in contrast to systolic data flow, where most data movement occurs through local interconnections between adjacent VMMs. Figures 11 (a) and (b) respectively show the computation cycle and normalized latency of both systolic and shared bus structures for different numbers of VMMs within a Tile. The computation cycle is defined as the proportion of VMM computation time to the total computation time period. As shown, the computation cycle decreases sharply with an increase in the size of the VMM array due to data movement through the shared bus. In contrast, the computation cycle of the systolic dataflow remains relatively stable because most data movement occurs through local communications. The computation cycle has been improved by 62% for 4×4 VMM array. Additionally, Figure 11 (b) illustrates the normalized latency, which is the latency for performing a convolutional layer with a specific input batch size of 16 × 16 and full utilization of VMMs to store weight parameters. There is a greater latency degradation for the shared bus structure compared to the systolic dataflow. This indicates that the systolic dataflow demonstrates better performance compared to the shared bus structure, making it more suitable for matrix multiplication applications.

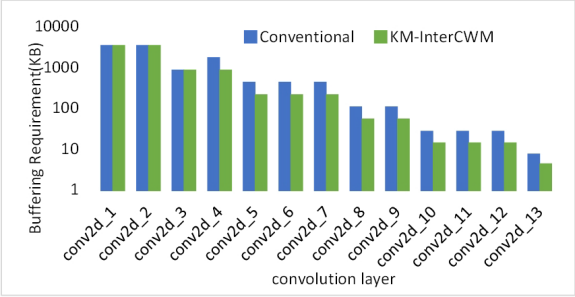


FIGURE 12. Buffering requirements per Tile for different layers of the VGG network using the proposed KM-InterCWM and conventional weight mapping schemes.

TABLE 4. Data reuse of consecutive convolution windows for different filter sizes using the proposed CM-IntraCWM weight mapping.

stride	1			2		
filter	3×3	5×5	7×7	3×3	5×5	7×7
data reuse(%)	67	80	86	33	60	71

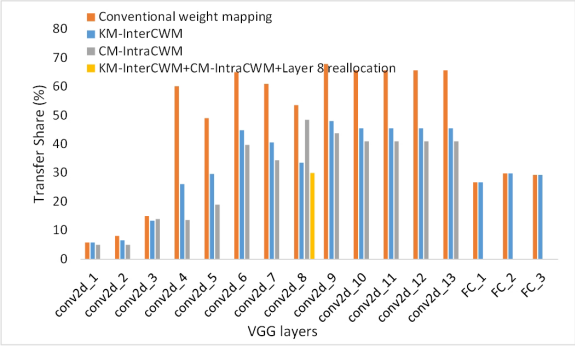


FIGURE 13. Contribution of data movement to inference delay across different layers of VGG-19 in SCiMA.

C. IMPACT OF PROPOSED WEIGHT MAPPING ON BUFFER REQUIREMENTS, DATA REUSE

Figure 12 illustrates that the proposed weight mapping reduces the buffering requirement by 57% for layer 3 and beyond of the largest convolutional layers in our benchmarks compared to the conventional weight mapping adopted by state-of-the-art architectures. The buffering requirement represents the amount of input data transfer needed for each Tile to perform computations. For the first three layers, this requirement remains unchanged because the number of input channels is fewer than the tile's processing capacity. Minimizing the buffering requirement, in turn, enhances both throughput and latency.

Table 4 illustrates the data reuse for different kernel sizes used in the VGG, AlexNet, and ResNet models. As shown, CM-IntraCWM enhances data reuse by 66% for a filter size of 3×3 and a stride of 1, without duplicating weights on the crossbar. Additionally, data reuse further increases for larger kernel sizes and smaller strides.

D. DATA MOVEMENT SHARE, INFERENCE DELAY, AND POWER IN SCIMA

Figure 13 illustrates the contribution of data transfer to inference delay in VGG's convolutional layers under different weight mapping strategies: conventional weight mapping used in state-of-the-art architectures, KM-InterCWM, and KM-InterCWM+CM-IntraCWM. These results were obtained by setting the data transfer rate between components affected by weight mapping to zero and then using actual transfer rates in SystemC simulations. Simulation results indicate that, with conventional weight mapping, data transfer accounts for 64% of the inference delay in intermediate layers (4-13). The KM-InterCWM approach significantly reduces this share to 37%, while KM-InterCWM+CM-IntraCWM further lowers it to 33%—except for layer 8, where the transfer share unexpectedly increases. Timing analysis of Tiles mapped to this layer reveals that faster output generation from adjacent Tiles under KM-InterCWM+CM-IntraCWM mapping increases NOC traffic, raising data transfer latency of packets through NOC. This highlights the need for a SystemC-based evaluation framework to analyze hardware performance under real workloads. The issue was resolved by reallocating layer 8 and alleviating the traffic of surrounding Tiles and the result has been reported in the Figure 13. For fully connected layers, both conventional and proposed weight mappings yield identical data transfer shares.

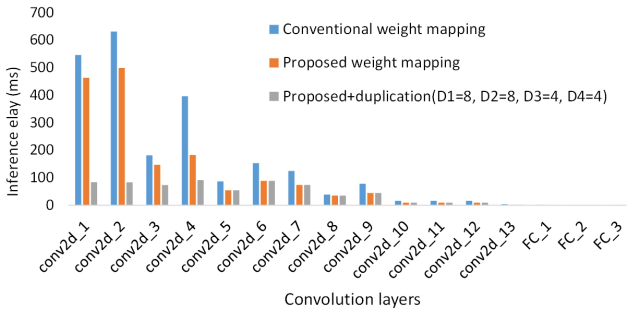


FIGURE 14. Inference delay for different layers of VGG in SCiMA.

Figure 14 compares inference delays for conventional and proposed weight mappings. Although the first four layers add just 0.2% area overhead, they heavily impact throughput. The proposed mapping reduces inference delay by 43% for layers 4 and beyond but only 15% for the first three layers. To address this, we duplicated hardware for early layers, assigning specific tiles and parallelizing input batches. Layers 1-4 were duplicated 8, 8, 4, and 4 times, respectively. Simulations show a significant inference delay reduction for initial layers, balancing performance with just 0.9% area overhead for VGG16. Unlike ISSAC, which replicates weights across all layers with 8, 16, and 32 chip configurations, our selective duplication efficiently enhances throughput.

Figure 15 presents the normalized power of DCNN benchmarks on SCiMA and Sparse SCiMA using conventional (CWM) and proposed (PWM) weight mappings. For VGG16, VGG19, and AlexNet, the pruning factor (PF) is set to 0.3, while ResNet-18 remains unpruned due to its convolution dominance. SparseSCiMA+CWM reduces power by 31% on average for VGG16 and VGG19. Although employing only sparsity lowers power more than employing only PWM for

TABLE 5. Comparison of SCiMA's hardware with other works.

	SCiMA 16@8	Sparse SCiMA 16@8	Sparse SCiMA 8@8	Domino [13] 8@8	ISAAC [5] 16@16	FORMS [12] 16@8	TM-CIM [35] 8@4
Technology	32			45	32	32	65
Benchmarks	VGG, AlexNet, RESNET			VGG,RESNET	VGG, MSRA	VGG,RESNET	VGG
Dataset	ImageNet						
Crossbar type (Size, cell storage)	ReRAM (128, 1)			ReRAM (256, 1)	ReRAM (128, 2)	ReRAM (128, 2)	ReRAM (256, 1)
Area(mm^2)	647.6	384.5	216.6	381	632	693	117.39
Power(W)	420.8	243.9	187.5	-	526.4	534.4	-
PE(OPs/W)	809G	1.32T	3.4T	9.3T	627G	526.68G	7.35T
CE($\frac{OPs}{mm2\times s}$)	524.3G	838.3G	2.4T	100G	478G	368.2G	283.3G
$\frac{infer.}{s}$	11	11	18.7	53	8	8.96	15.6

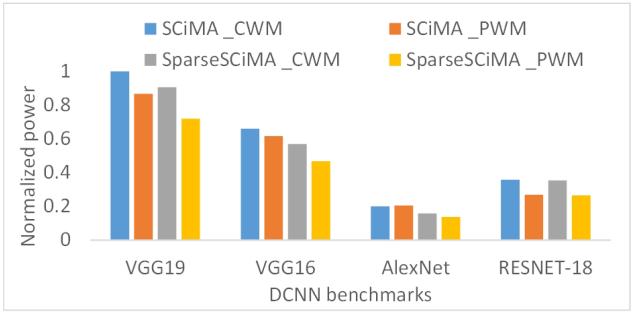


FIGURE 15. Normalized power relative to SCiMA-CWM executing VGG19 for SCiMA-CWM, SCiMA-PWM, SparseSCiMA-CWM, and SparseSCiMA-PWM across DCNN benchmarks.

VGG16, the opposite is true for VGG19 due to its lower dense-layer dominance. Since 95% of ResNet-18's weights are in convolution layers, sparsity has no effect, whereas PWM alone reduces power by 34%. Additionally, PWM alone cuts power by 11% on average for VGG16 and VGG19. Notably, power reduction is less than the transfer share reduction in VGG networks because while convolution power decreases, dense layers which are dominant consume more due to obtained higher throughput.

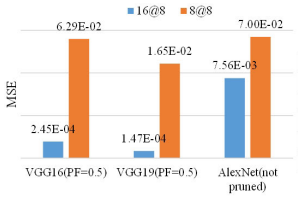
E. COMPARISON WITH STATE-OF-THE-ART ARCHITECTURES

Table 5 compares the characteristics of SCiMA and Sparse SCiMA—both in 16@8 and 8@8 configurations—with state-of-the-art designs like ISAAC, Domino, FORMS, and TM-CiM for VGG16 and VGG19 implementation. Among these, FORMS employs layer-level pruning, removing entire rows and columns of the weight matrix. In our work, the Sparse SCiMA 8@8 variant retains the same 48 × 48 Tile array as the 16@8 configuration but reduces SRAM and buffer sizes by half for activation storage. Unlike prior architectures, which were evaluated analytically without real traffic considerations, this work develops a SystemC model to assess hardware performance under real workloads.

Among architectures using 16-bit activations (16@8), SCiMA demonstrates comparable power and area metrics to FORMS and ISAAC. This is notable since both FORMS and ISAAC utilize eDRAM, which is more area-efficient, whereas SCiMA employs SRAM. Additionally, while ISAAC and FORMS use two bits per cell in their crossbars, SCiMA employs single-bit cells, requiring a greater number of crossbars to achieve equivalent precision. Despite this, SCiMA's overall efficiency suggests that its peripheral memory system is more optimized than that of FORMS and ISAAC, effectively mitigating the area and power costs associated with the increased number of crossbars. Applying a pruning method that reduces dense layer parameters by 50% (PF = 0.03), Sparse SCiMA achieves a 42% reduction in area and a 56% decrease in power consumption.

SCiMA improves Power Efficiency (PE) and Computational Efficiency (CE) by 27% and 17%, respectively, compared to ISAAC and FORMS. These gains stem from enhancements in computation cycles, data transfer share, buffer requirements, and data reuse through the proposed systolic dataflow and weight mapping strategies compared to counterpart methods employed in state-of-the-art architectures. Sparse SCiMA further amplifies these benefits at the cost of 0.8% accuracy loss, reaching a PE of 3.4T OPs/W, compared to 627G OPs/W for ISAAC and 526G OPs/W for FORMS. In terms of throughput, SCiMA and Sparse SCiMA achieve 11 inferences per second, surpassing ISAAC's 8 and FORMS' 8.96 inferences per second. Our framework can calculate dynamic power during the execution of real application workloads, reporting power consumption of 66.1W for VGG19 and 43W for VGG16.

For architectures using 8-bit activations (8@8), Sparse SCiMA has an area of 216.6 mm^2, which is lower than Domino's 381 mm^2 but higher than TM-CiM's 117.39 mm^2. Domino employs two input and output routers per crossbar along with the necessary buffering resources, whereas in this work, a single router is shared among four tiles, and each 4×4 VMM within a tile is supported by a dedicated SRAM. Its power efficiency in the 8@8 configuration reaches 3.4T OPs/W, compared to 9.3T OPs/W for Domino and 7.35T OPs/W for TM-CiM. While Domino and TM-CiM use lower technology nodes, both employ larger 256 × 256 crossbar arrays for analog computations and a TM-CiM 4-bit weight width, reducing the need for peripheral digital circuits. Despite these differences, Sparse SCiMA remains competitive. Nevertheless, Sparse SCiMA achieves a high inference throughput of 18.7 inferences per second, surpassing TM-CiM's 15.6 but lower than Domino's 53 inferences per second. This improvement is attributed to enhancements in computation cycles, data reuse, and buffer requirements.



(a)

SystemC vs RTL	
Simulation time	Timing precision
58.7×	99.75%
Simulation time of matrix-based emulation vs	
For-loop	Neuron-vectorized
~150×	~8×

(b)

FIGURE 16. (a) MSE of the hardware relative to quantized model, (b) Comparison of simulation time and SystemC precision.

F. SIMULATION TIME OF VIRUTAL FRAMEWORK

Figure 16(a) presents the MSE from emulated analog computations based on [38] and the quantized pruned model, across various CNN benchmarks using 2000 ImageNet instances. The study considers both 8-bit and 16-bit activation widths. LRS and HRS were set to 3KΩ and 10KΩ, respectively, with a 15% conductance mapping error [17]. The analog computation yields an MSE of $2.54e^{-4}$ for VGG16, $1.47e^{-4}$ for VGG19, and a significantly higher $7.56e^{-3}$ for AlexNet. These results highlight the sharp MSE increase when reducing activation width to 8 bits. This paper primarily focuses on efficient emulation of memristor crossbars for DCNNs rather than hardware performance evaluation. Figure 16(b) shows that the proposed matrix-based emulation approach accelerates simulations significantly, 150× faster than a for-loop implementation and 8× faster than a neuron-vectorized method approximately. These simulation times have been recorded over multiple runs on the Google Colab TPU.

To assess SystemC modeling against RTL simulation for SCiMA, we compared simulation speed and performance metric accuracy (Table of Figure 16(b)). RTL simulations were conducted using ModelSim under identical conditions. SystemC achieved a 58.7× speedup while maintaining 99.7% accuracy in inference delay evaluation, demonstrating its efficiency and precision in modeling complex architectures like SCiMA. Among the frameworks, none have reported the MSE for DCNNs at the scale of VGG16. Additionally, only MNSIM has reported hardware latency results for VGG16.

VIII. CONCLUSION

This paper introduced a programmable Computation-in-Memory (CiM)-based Systolic Accelerator (SCiMA), featuring a scalable Tile-based architecture optimized for convolutional neural network (CNN) applications. By leveraging novel weight mapping and pruning strategies, SCiMA enhances data transfer and data reuse efficiency, leading to increased computational and power efficiency. Additionally, a virtual system-level framework was proposed that includes a compiler for mapping CNNs to configurable hardware, an emulator for assessing analog computation reliability, and a functional SystemC model for architectural exploration and evaluation. For future work, more detailed electrical properties of CiM-based hardware is integrated into our simulation framework.

REFERENCES

[1] N. Verma, H. Jia, H. Valavi, Y. Tang, M. Ozatay, L.-Y. Chen, B. Zhang, and P. Deaville, "In-memory computing: Advances and prospects," *IEEE Solid State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Summer. 2019.

[2] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, Jun. 2019.

[3] G. Singh, L. Chelini, S. Corda, A. J. Awan, S. Stuijk, R. Jordans, H. Corporaal, and A.-J. Boonstra, "A review of near-memory computing architectures: Opportunities and challenges," in *Proc. 21st Euromicro Conf. Digit. Syst. Design (DSD)*, Prague, Czech Republic, Aug. 2018, pp. 608–617.

[4] M. A. Hanif, A. Manglik, and M. Shafique, "Resistive crossbar-aware neural network design and optimization," *IEEE Access*, vol. 8, pp. 229066–229085, 2020.

[5] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 14–26.

[6] R. Liu, X. Peng, X. Sun, W. S. Khwa, X. Si, J. J. Chen, J. F. Li, M. F. Chang, and S. Yu, "Parallelizing SRAM arrays with customized bit-cell for binary neural networks," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jun. 2018, pp. 1–6.

[7] F. Gao, G. Tziantzioulis, and D. Wentzlaff, "ComputeDRAM: In-memory compute using Off-the-Shelf DRAMs," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO 52)*, vol. 2019, New York, NY, USA, Oct. 2019, pp. 100–113.

[8] B. Li, B. Yan, and H. Li, "An overview of in-memory processing with emerging non-volatile memory for data-intensive applications," in *Proc. Great Lakes Symp. VLSI (GLSVLSI)*, New York, NY, USA, May 2019, pp. 381–386.

[9] H. Jin, C. Liu, H. Liu, R. Luo, J. Xu, F. Mao, and X. Liao, "ReHy: A ReRAM-based digital/analog hybrid PIM architecture for accelerating CNN training," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2872–2884, Nov. 2022, doi: 10.1109/TPDS.2021.3138087.

[10] C. Luo, J. Diao, and C. Chen, "FullReuse: A novel ReRAM-based CNN accelerator reusing data in multiple levels," in *Proc. IEEE 5th Int. Conf. Integr. Circuits Microsystems (ICICM)*, Nanjing, China, Oct. 2020, pp. 177–183.

[11] X. Peng, R. Liu, and S. Yu, "Optimizing weight mapping and data flow for convolutional neural networks on RRAM based processing-in-memory architecture," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2019, pp. 1–5.

[12] G. Yuan, P. Behnam, Z. Li, A. Shafiee, S. Lin, X. Ma, H. Liu, X. Qian, M. N. Bojnordi, Y. Wang, and C. Ding, "FORMS: Fine-grained polarized ReRAM-based in-situ computation for mixed-signal DNN accelerator," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, Jun. 2021, pp. 265–278.

[13] K. Zhou, Y. He, R. Xiao, J. Liu, and K. Huang, "A customized NoC architecture to enable highly localized computing-on-the-move DNN dataflow," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1692–1696, Mar. 2022.

[14] X. Qiao, X. Cao, H. Yang, L. Song, and H. Li, "AtomLayer: A universal ReRAM-based CNN accelerator with atomic layer computation," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jun. 2018, pp. 1–6.

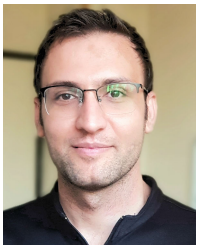
[15] M. Mao, X. Peng, R. Liu, J. Li, S. Yu, and C. Chakrabarti, "MAX2: An ReRAM-based neural network accelerator that maximizes data reuse and area utilization," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 398–410, Jun. 2019.

[16] J. Rhe, S. Moon, and J. H. Ko, "VW-SDK: Efficient convolutional weight mapping using variable windows for processing-in-memory architectures," in *Proc. Design, Autom. Test Eur. (DATE)*, Mar. 2022, vol. 2022, no. 1, pp. 214–219.

[17] M. Hu, J. P. Strachan, Z. Li, E. M. Grafals, N. Davila, C. Graves, S. Lam, N. Ge, J. J. Yang, and R. S. Williams, "Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication," in *Proc. 53nd ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, Austin, TX, USA, Jun. 2016, pp. 1–6.

[18] L. Kull, T. Toifl, M. Schmatz, P. A. Francese, C. Menolfi, M. Brändli, M. Kossel, T. Morf, T. M. Andersen, and Y. Leblebici, "A 3.1 mW 8b 1.2 GS/s single-channel asynchronous SAR ADC with alternate comparators for enhanced speed in 32 nm digital SOI CMOS," *IEEE J. Solid-State Circuits*, vol. 48, no. 12, pp. 3049–3058, Dec. 2013.

[19] P.-Y. Chen, X. Peng, and S. Yu, "NeuroSim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 12, pp. 3067–3080, Dec. 2018.
[20] Y.-C. Lo, C.-C. Yeh, J.-S. Wu, C.-C. Wang, Y.-C. Tsai, W.-C. Ting, and R.-S. Liu, "ISSA: Input-skippable, set-associative computing-in-memory (SA-CIM) architecture for neural network accelerators," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, San Diego, CA, USA, Oct. 2022, pp. 1–9.
[21] Y.-C. Lo and R.-S. Liu, "Morphable CIM: Improving operation intensity and depthwise capability for SRAM-CIM architecture," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jul. 2023, pp. 1–6.
[22] J. N. Rohan and J. P. Kulkarni, "Systolic-RAM: Scalable direct convolution using in-memory data movement," *IEEE Solid-State Circuits Lett.*, vol. 5, pp. 142–145, 2022.
[23] X. Chen, R. Guo, Z. Yue, Y. Hu, L. Liu, S. Wei, and S. Yin, "A systolic Computing-in-Memory array based accelerator with predictive early activation for spatiotemporal convolutions," in *Proc. IEEE 5th Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, Hangzhou, China, Jun. 2023, pp. 1–5.
[24] Y. Zhan, W.-H. Yu, K.-F. Un, R. P. Martins, and P.-I. Mak, "GSLP-CIM: A 28-nm globally systolic and locally parallel CNN/transformer accelerator with scalable and reconfigurable eDRAM compute-in-memory macro for flexible dataflow," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 4, pp. 1657–1667, Apr. 2025.
[25] S. Wang, Z. Li, Y. Ma, and Y. Kang, "SysCIM: A heterogeneous chip architecture for high-efficiency CNN training at edge," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 4, pp. 1–14, Apr. 2025.
[26] D. Liu, H. Zhou, W. Mao, J. Liu, Y. Han, C. Man, Q. Wu, Z. Guo, M. Huang, S. Luo, M. Lv, Q. Chen, and H. Yu, "An energy-efficient mixed-bit CNN accelerator with column parallel readout for ReRAM-based in-memory computing," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 4, pp. 821–834, Dec. 2022.
[27] J. Kndtel, M. Fritscher, D. Reiser, D. Fey, M. Breiling, and M. Reichenbach, "A model-to-circuit compiler for evaluation of DNN accelerators based on systolic arrays and multibit emerging memories," in *Proc. 9th Int. Conf. Modern Circuits Syst. Technol. (MOCAST)*, Bremen, Germany, Sep. 2020, pp. 1–6.
[28] S. Thakur and S. Bodapati, "Ternary systolic array architecture for matrix multiplication in CNFET-memristor technology," in *Proc. IEEE Asia–Pacific Conf. Circuits Syst. (APCCAS)*, Hyderabad, India, Nov. 2023, pp. 45–49.
[29] N. Challapalle, S. Rampalli, M. Chandran, G. S. Kalsi, S. Subramoney, J. Sampson, and V. Narayanan, "PSB-RNN: A processing-in-memory systolic array architecture using block circulant matrices for recurrent neural networks," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Grenoble, France, Mar. 2020, pp. 180–185.
[30] M. R. Haq Rashed, S. Thijssen, S. K. Jha, H. Zheng, and R. Ewetz, "Path-based processing using in-memory systolic arrays for accelerating data-intensive applications," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, San Francisco, CA, USA, Oct. 2023, pp. 1–9.
[31] J. Yue, Y. Liu, Z. Yuan, X. Feng, Y. He, W. Sun, Z. Zhang, X. Si, R. Liu, Z. Wang, M.-F. Chang, C. Dou, X. Li, M. Liu, and H. Yang, "STICKER-IM: A 65 nm Computing-in-Memory NN processor using block-wise sparsity optimization and inter/intra-macro data reuse," *IEEE J. Solid-State Circuits*, vol. 57, no. 8, pp. 2560–2573, Aug. 2022.
[32] T. Andrulis, J. S. Emer, and V. Sze, "CiMLoop: A flexible, accurate, and fast compute-in-memory modeling tool," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, May 2024, pp. 10–23.
[33] M. Imani, S. Gupta, Y. Kim, and T. Rosing, "FloatPIM: In-memory acceleration of deep neural network training with high precision," in *Proc. ACM/IEEE 46th Annu. Int. Symp. Comput. Archit. (ISCA)*, Phoenix, AZ, USA, Jun. 2019, pp. 802–815.
[34] L. Song, X. Qian, H. Li, and Y. Chen, "PipeLayer: A pipelined ReRAM-based accelerator for deep learning," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2017, pp. 541–552.
[35] R. Xiao, W. Jiang, and P. Y. Chee, "An energy efficient time-multiplexing computing-in-memory architecture for edge intelligence," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 8, pp. 111–118, 2022.
[36] G. Zhang, R. Zhang, R. Wang, and S. Zhu, "A systolic array-based scheduling strategy for sparse CNN accelerators," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 71, no. 3, pp. 1436–1440, Mar. 2024.
[37] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to DNN accelerator evaluation," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, Mar. 2019, pp. 304–315.
[38] S. Vahdat, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Interstice: Inverter-based memristive neural networks discretization for function approximation applications," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1578–1588, Jul. 2020.
[39] Z. Zhu, H. Sun, T. Xie, Y. Zhu, G. Dai, L. Xia, D. Niu, X. Chen, X. S. Hu, Y. Cao, Y. Xie, H. Yang, and Y. Wang, "MNSIM 2.0: A behavior-level modeling tool for Processing-In-Memory architectures," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 11, pp. 4112–4125, Nov. 2023.
[40] F. Liu, W. Zhao, Z. Wang, Y. Chen, X. Liang, and L. Jiang, "ERA-BS: Boosting the efficiency of ReRAM-based PIM accelerator with fine-grained bit-level sparsity," *IEEE Trans. Comput.*, vol. 73, no. 9, pp. 2320–2334, Sep. 2024.
[41] S. Qu, B. Li, Y. Wang, and L. Zhang, "ASBP: Automatic structured bit-pruning for RRAM-based NN accelerator," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Dec. 2021, pp. 745–750.
[42] J. Meng, L. Yang, X. Peng, S. Yu, D. Fan, and J.-S. Seo, "Structured pruning of RRAM crossbars for efficient in-memory computing acceleration of deep neural networks," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 5, pp. 1576–1580, May 2021.
[43] T.-H. Yang, H.-Y. Cheng, C.-L. Yang, I.-C. Tseng, H.-W. Hu, H.-S. Chang, and H.-P. Li, "Sparse ReRAM engine: Joint exploration of activation and weight sparsity in compressed neural networks," in *Proc. ACM/IEEE 46th Annu. Int. Symp. Comput. Archit. (ISCA)*, Phoenix, AZ, USA, Jun. 2019, pp. 236–249.
[44] L. Zheng, H. Liu, Y. Huang, D. Chen, C. Liu, H. He, X. Liao, H. Jin, and J. Xue, "A flexible yet efficient DNN pruning approach for crossbar-based Processing-in-Memory architectures," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 11, pp. 3745–3756, Nov. 2022.
[45] S. Yu, L. Zhang, J. Wang, J. Yue, Z. Yuan, X. Li, H. Yang, and Y. Liu, "High area/energy efficiency RRAM CNN accelerator with pattern-pruning-based weight mapping scheme," in *Proc. IEEE 10th Non-Volatile Memory Syst. Appl. Symp. (NVMSA)*, Beijing, China, Aug. 2021, pp. 1–6.
[46] C. Chu, Y. Wang, Y. Zhao, X. Ma, S. Ye, Y. Hong, X. Liang, Y. Han, and L. Jiang, "PIM-prune: Fine-grain DCNN pruning for crossbar-based process-in-memory architecture," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jul. 2020, pp. 1–6.
[47] J. Li, Y. Fu, C.-W. Sham, and S. L. Ma, "A framework for mapping convolutional neural network onto memristor crossbars," in *Proc. IEEE 13th Global Conf. Consum. Electron. (GCCE)*, Kitakyushu, Japan, Oct. 2024, pp. 1390–1393.



REZGAR SADEGHI (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from the University of Kurdistan (UOK), Sanandaj, Iran, in 2015 and 2017, respectively. He is currently pursuing the Ph.D. degree with the University of Tehran (UT). He is a Research Assistant with UT. His current research interests include neuromorphic computing, accelerator design, virtual frameworks, and interconnect reliability. He also has industrial experience on hardware-in-the-loop (HIL) and AI-based embedded systems with automotive Companies.



ZAINALABEDIN NAVABI (Life Senior Member, IEEE) received the B.S. degree from The University of Texas at Austin, Austin, TX, USA, in 1975, and the M.S. and Ph.D. degrees from The University of Arizona, Tucson, AZ, USA, in 1978 and 1981, respectively. He is currently a Professor of electrical and computer engineering with the University of Tehran, Tehran, Iran, and an Adjunct Professor with Worcester Polytechnic Institute, Worcester, MA, USA. He has authored ten books in various aspects of design and test with hardware description languages (HDLs). He has written numerous articles on HDLs, design automation, and digital system test. His current research interests include high-level design and description methodologies, digital system testing, and design and definition of HDLs.